МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

[Полное наименование образовательной организации]

Кафедра [наименование кафедры]

Лабораторная работа № 4

Анализ алгоритмов последовательного, двоичного и интерполяционного поиска

Дисциплина: Теория алгоритмов

Направление подготовки: 38.03.05 «Бизнес-информатика»

Язык реализации: Python 3.10+

Вариант: 14

Выполнил: студент группы [номер группы] [ФИО]

Проверил: [должность, ФИО преподавателя]

[Город], 2026

1. Цель работы и задание варианта

1.1. Цель работы

Изучить алгоритмы поиска элемента в массиве, экспериментально сопоставить их трудоёмкость с теоретическими оценками и определить условия, при которых предварительная сортировка данных оправдана.

1.2. Задание варианта 14

Сравнить последовательный, двоичный и интерполяционный поиск на упорядоченных массивах, ключи которых образуют арифметическую прогрессию, при размерах $n = 5 \cdot 10^3,\ 5 \cdot 10^4,\ 5 \cdot 10^5$. Число запросов $k = 2000$, запросы смешанные (50 % успешных, 50 % неуспешных). Выполнить дополнительное задание Б7: построить гистограмму числа сравнений по всем запросам для каждого алгоритма и сопоставить эмпирическое среднее с теоретическими значениями $\frac{n+1}{2}$ и $\log_2 n$.

Таблица 1. Параметры эксперимента

Параметр	Значение
Размеры массива n	$5\,000$; $50\,000$; $500\,000$
Распределение (код А)	$A[i] = c + d \cdot i$, принято $c = 13$, $d = 7$
Число запросов k	2000: 1000 успешных и 1000 неуспешных, порядок перемешан
Успешный запрос	ключ — случайный элемент массива
Неуспешный запрос	ключ $A[i] + r$, где i случайно, $r \in \{1, \dots, 6\}$: лежит строго между соседними элементами, поэтому отсутствует в массиве, но попадает в диапазон $[A[0];\ A[n-1]]$
Дополнительное задание	Б7 — гистограммы числа сравнений
Повторов замера	3 (берётся медиана среднего времени запроса)

Значения c и d в задании не заданы; результат от них не зависит, поскольку алгоритмы используют только порядок и разности ключей.

Что считается сравнением. Во всех трёх реализациях одно сравнение — одна проверка совпадения ключа `a[i] == q` (в двоичном и интерполяционном поиске — по одной на итерацию цикла). Фактически двоичный поиск выполняет на итерации до двух сравнений; принятое соглашение не меняет асимптотику и соответствует образцу из методических указаний.

2. Описание алгоритмов и теоретические оценки

2.1. Последовательный поиск

Элементы просматриваются подряд. При успешном поиске с равновероятным положением ключа среднее число сравнений равно $\frac{n+1}{2}$, при неуспешном всегда n. Для смеси запросов 50/50

$$C_{\text{лин}} = \frac{1}{2} \cdot \frac{n+1}{2} + \frac{1}{2} \cdot n = \frac{3n+1}{4} = \Theta(n).$$

2.2. Двоичный поиск

Применим только к упорядоченному массиву; отрезок на каждом шаге сокращается вдвое. В худшем случае $\lfloor \log_2 n \rfloor + 1$ сравнений. Для сопоставления с экспериментом полезны точные средние. Дерево двоичного поиска сбалансировано (все уровни, кроме последнего, заполнены). Пусть $h = \lfloor \log_2 n \rfloor + 1$ и

$$I = \sum_{j=0}^{h-2} j \cdot 2^j + (h-1)\left(n - 2^{h-1} + 1\right)$$

— суммарная глубина узлов (корень имеет глубину 0). Тогда

$$C_{\text{усп}} = \frac{I+n}{n}, \qquad C_{\text{неусп}} = \frac{I+2n}{n+1}.$$

Эти формулы проверяются в программе перебором всех ключей малых массивов (функция `self_test`). Для $n = 5 \cdot 10^5$ получаем $C_{\text{усп}} \approx 17{,}95$ и $C_{\text{неусп}} \approx 18{,}95$, то есть примерно $\log_2 n - 1$ и $\log_2 n$ при $\log_2 n \approx 18{,}93$. Поэтому $\log_2 n$ — оценка сверху для среднего числа сравнений, а не точное значение.

2.3. Интерполяционный поиск

Пробная позиция

$$mid = lo + \left\lfloor \frac{(q - A[lo]) \cdot (hi - lo)}{A[hi] - A[lo]} \right\rfloor.$$

Для равномерно распределённых **случайных** ключей ожидаемое число сравнений $O(\log \log n)$. Распределение A — частный, идеальный случай: ключи расположены строго линейно. Пусть $q = A[j]$ и $lo \le j \le hi$. Тогда

$$mid = lo + \left\lfloor \frac{d\,(j - lo)(hi - lo)}{d\,(hi - lo)} \right\rfloor = j,$$

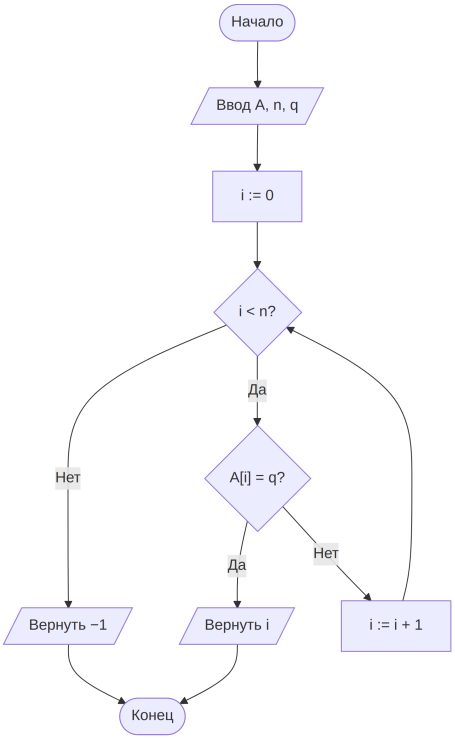
и ключ находится **с первого сравнения**. Для неуспешного ключа $q = A[j] + r$, $0 < r < d$, дробная добавка равна $\frac{r}{d} < 1$, поэтому снова $mid = j$; так как $A[j] < q$, берётся $lo = j + 1$, и тогда $A[lo] = A[j] + d > q$ — условие цикла нарушено, поиск завершается результатом «−1», также после одного сравнения. Итак, на данных варианта 14 ожидается ровно **одно сравнение при любых** n, то есть $\Theta(1)$ — лучше оценки $\log \log n$.

2.4. Сводка ожиданий для варианта 14

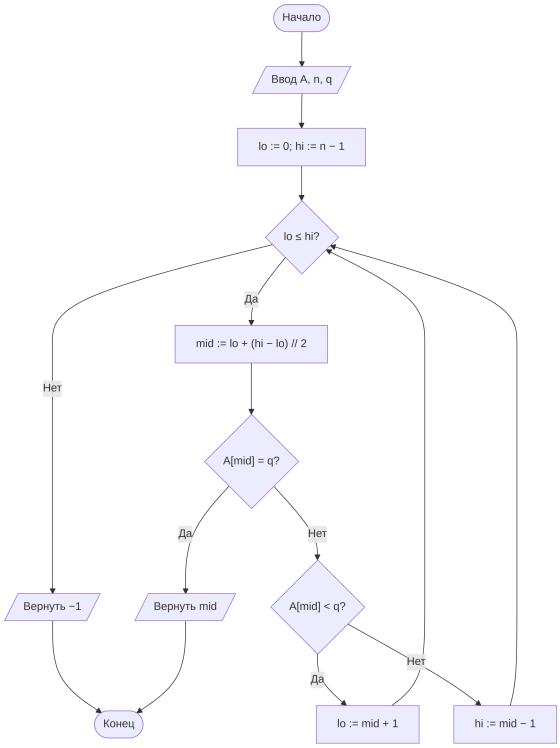
Алгоритм	Ожидаемое число сравнений (смесь 50/50)
Последовательный	$\frac{3n+1}{4}$
Двоичный	$\approx \log_2 n - 0{,}5$ (среднее точных значений для успешного и неуспешного поиска), не более $\lfloor \log_2 n \rfloor + 1$
Интерполяционный	1

3. Блок-схемы алгоритмов

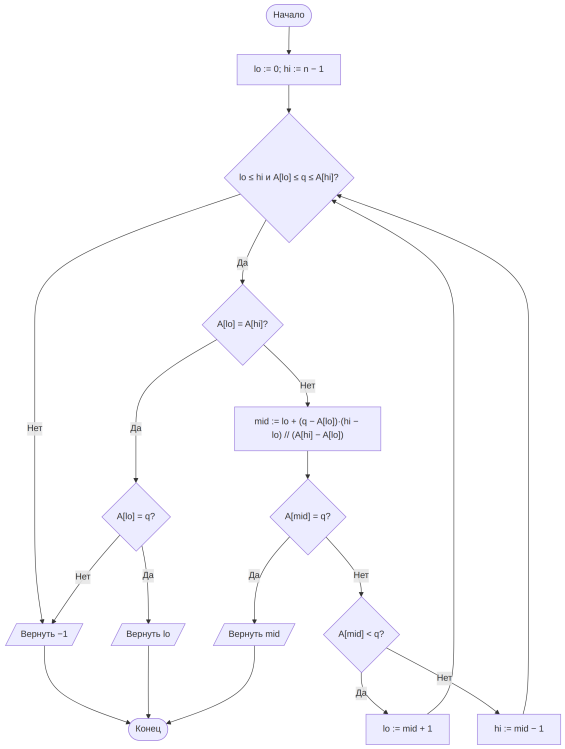
3.1. Последовательный поиск



3.2. Двоичный поиск



3.3. Интерполяционный поиск



4. Листинг программы

Программа содержит реализации трёх алгоритмов, генерацию данных и запросов, проверку корректности (каждый ответ сверяется с известным фактом наличия ключа), замеры времени, сбор числа сравнений по каждому запросу и построение графиков.

```
"""
Лабораторная работа № 4. Вариант 14.
Последовательный, двоичный и интерполяционный поиск.

Распределение ключей: A (арифметическая прогрессия A[i] = c + d*i).
Размеры массива: n = 5·10^3, 5·10^4, 5·10^5.
Запросов k = 2000, смешанные: 50 % успешных и 50 % неуспешных.
Дополнительное задание Б7: гистограммы числа сравнений по всем запросам.
"""
import gc
import json
import math
import platform
import random
import statistics
import time

C0, D = 13, 7                       # параметры прогрессии A[i] = C0 + D*i
SIZES = [5_000, 50_000, 500_000]
K = 2000                            # число запросов (1000 успешных + 1000 неуспешных)
REPEATS = 3                         # число повторов замера (берётся медиана)


# ---------------------------------------------------------------- алгоритмы
def linear_search(a, q):
    """Последовательный поиск. Возвращает (индекс, число сравнений)."""
    for i, value in enumerate(a):
        if value == q:
```

```
            return i, i + 1
    return -1, len(a)


def binary_search(a, q):
    """Двоичный поиск в упорядоченном массиве (одно сравнение a[mid] == q на шаг)."""
    lo, hi, cmps = 0, len(a) - 1, 0
    while lo <= hi:
        mid = lo + (hi - lo) // 2
        cmps += 1
        if a[mid] == q:
            return mid, cmps
        if a[mid] < q:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1, cmps


def interpolation_search(a, q):
    """Интерполяционный поиск в упорядоченном массиве."""
    lo, hi, cmps = 0, len(a) - 1, 0
    while lo <= hi and a[lo] <= q <= a[hi]:
        if a[lo] == a[hi]:
            cmps += 1
            return (lo, cmps) if a[lo] == q else (-1, cmps)
        mid = lo + (q - a[lo]) * (hi - lo) // (a[hi] - a[lo])
        cmps += 1
        if a[mid] == q:
            return mid, cmps
        if a[mid] < q:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1, cmps


ALGORITHMS = {"Последовательный": linear_search,
              "Двоичный": binary_search,
              "Интерполяционный": interpolation_search}


# ---------------------------------------------------------------------- данные
def make_data(n):
    """Арифметическая прогрессия из n элементов: A[i] = C0 + D*i."""
    return [C0 + D * i for i in range(n)]


def make_queries(a, k, rng):
    """k запросов: половина успешных, половина неуспешных.
    Неуспешный ключ лежит строго между двумя соседними элементами массива,
    поэтому он заведомо отсутствует, но попадает в диапазон [A[0]; A[n-1]]."""
    n = len(a)
    hits = [a[rng.randrange(n)] for _ in range(k // 2)]
    misses = [a[rng.randrange(n - 1)] + rng.randint(1, D - 1) for _ in range(k // 2)]
    queries = [(q, True) for q in hits] + [(q, False) for q in misses]
    rng.shuffle(queries)
    return queries
```

```python
# ---------------------------------------------------------------- замеры
def benchmark(search_func, data, keys):
    """Среднее время одного запроса, с, и среднее число сравнений."""
    gc.collect()
    start = time.perf_counter()
    total_cmps = 0
    for q in keys:
        idx, cmps = search_func(data, q)
        total_cmps += cmps
    elapsed = time.perf_counter() - start
    return elapsed / len(keys), total_cmps / len(keys)


def collect_comparisons(search_func, data, queries):
    """Число сравнений по каждому запросу (для гистограмм) + проверка корректности."""
    result = []
    for q, present in queries:
        idx, cmps = search_func(data, q)
        assert (idx != -1) == present, "поиск вернул неверный результат"
        if present:
            assert data[idx] == q
        result.append(cmps)
    return result


# ---------------------------------------------------------------- теория
def binary_expected(n):
    """Точные средние числа сравнений двоичного поиска (одно сравнение на шаг):
    успешный и неуспешный поиск при равновероятных исходах.
    Дерево поиска сбалансировано: все уровни, кроме последнего, заполнены."""
    h = n.bit_length()                                  # floor(log2 n) + 1
    internal_path = sum(j * 2 ** j for j in range(h - 1)) + (h - 1) * (n - (2 ** (h - 1) - 1))
    succ = (internal_path + n) / n
    unsucc = (internal_path + 2 * n) / (n + 1)
    return succ, unsucc


def theory(n):
    succ_bin, unsucc_bin = binary_expected(n)
    return {
        "lin_hit": (n + 1) / 2, "lin_miss": n, "lin_mix": (3 * n + 1) / 4,
        "bin_log2": math.log2(n), "bin_hit": succ_bin, "bin_miss": unsucc_bin,
        "bin_mix": (succ_bin + unsucc_bin) / 2,
        "int_loglog": math.log2(math.log2(n)),
    }


def self_test():
    """Проверка формулы binary_expected перебором всех ключей малого массива."""
    for n in (1, 2, 7, 10, 100, 1000):
        a = list(range(0, 2 * n, 2))
        hit = sum(binary_search(a, x)[1] for x in a) / n
        miss = sum(binary_search(a, x)[1] for x in range(-1, 2 * n, 2)) / (n + 1)
        s, u = binary_expected(n)
        assert abs(hit - s) < 1e-9 and abs(miss - u) < 1e-9, (n, hit, s, miss, u)


# ---------------------------------------------------------------- график
def make_plots(res):
    import matplotlib
```

```
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    names = list(ALGORITHMS)
    ns = [r["n"] for r in res]
    markers = {"Последовательный": "o", "Двоичный": "s", "Интерполяционный": "^"}

    # Рис. 1. Среднее время запроса
    plt.figure(figsize=(6.4, 4.2))
    for name in names:
        plt.plot(ns, [r["time"][name] * 1e6 for r in res], marker=markers[name], label=name)
    plt.xscale("log"); plt.yscale("log")
    plt.xlabel("n (логарифмическая шкала)"); plt.ylabel("среднее время запроса, мкс")
    plt.title("Среднее время одного запроса"); plt.grid(True, which="both", alpha=.3)
    plt.legend(); plt.tight_layout(); plt.savefig("img/lr4_time.png", dpi=170); plt.close()

    # Рис. 2. Среднее число сравнений
    plt.figure(figsize=(6.4, 4.2))
    for name in names:
        plt.plot(ns, [r["cmps"][name] for r in res], marker=markers[name], label=name + " (опыт)")
    plt.plot(ns, [r["theory"]["lin_mix"] for r in res], "k--", lw=1, label=r"$(3n+1)/4$")
    plt.plot(ns, [r["theory"]["bin_log2"] for r in res], "k:", lw=1.2, label=r"$\log_2 n$")
    plt.xscale("log"); plt.yscale("log")
    plt.xlabel("n (логарифмическая шкала)"); plt.ylabel("среднее число сравнений")
    plt.title("Среднее число сравнений на запрос"); plt.grid(True, which="both", alpha=.3)
    plt.legend(fontsize=8); plt.tight_layout(); plt.savefig("img/lr4_cmps.png", dpi=170);
↪   plt.close()

    # Рис. 3. Гистограммы (строки — алгоритмы, столбцы — n)
    fig, axes = plt.subplots(3, len(res), figsize=(11, 8.2))
    for j, r in enumerate(res):
        for i, name in enumerate(names):
            ax = axes[i][j]
            data = r["all_cmps"][name]
            mean = statistics.mean(data)
            if name == "Последовательный":
                ax.hist(data, bins=40, color="#4c72b0")
                ax.set_yscale("log")
                ax.axvline(r["theory"]["lin_mix"], color="k", ls="--", lw=1.2,
                           label=f"теория (3n+1)/4 = {r['theory']['lin_mix']:.0f}")
            elif name == "Двоичный":
                lo, hi = min(data), max(data)
                ax.hist(data, bins=[x - .5 for x in range(lo, hi + 2)], color="#55a868")
                ax.axvline(r["theory"]["bin_log2"], color="k", ls=":", lw=1.4,
                           label=f"log₂n = {r['theory']['bin_log2']:.1f}")
            else:
                ax.hist(data, bins=[x - .5 for x in range(min(data), max(data) + 2)],
↪   color="#c44e52")
                ax.set_xlim(-1, 6)
            ax.axvline(mean, color="red", lw=1.4, label=f"опыт, среднее = {mean:.1f}")
            ax.set_title(f"{name}, n = {r['n']:,}".replace(",", " "), fontsize=9)
            ax.legend(fontsize=7)
            if j == 0:
                ax.set_ylabel("число запросов")
            if i == 2:
                ax.set_xlabel("число сравнений")
    plt.tight_layout(); plt.savefig("img/lr4_hist.png", dpi=150); plt.close()


# ---------------------------------------------------------------- main
if __name__ == "__main__":
```

```
    self_test()
    rng = random.Random(14)
    results = []
    print(f"Вариант 14: A[i] = {C0} + {D}*i, k = {K} (50/50), повторов {REPEATS}")
    for n in SIZES:
        data = make_data(n)
        queries = make_queries(data, K, rng)
        keys = [q for q, _ in queries]
        row = {"n": n, "time": {}, "cmps": {}, "all_cmps": {}, "stat": {}, "theory": theory(n)}
        print(f"\nn = {n}")
        for name, func in ALGORITHMS.items():
            runs = [benchmark(func, data, keys) for _ in range(REPEATS)]
            t = statistics.median(x[0] for x in runs)
            c = runs[0][1]
            per_query = collect_comparisons(func, data, queries)
            hit = [x for x, (_, p) in zip(per_query, queries) if p]
            miss = [x for x, (_, p) in zip(per_query, queries) if not p]
            row["time"][name], row["cmps"][name] = t, c
            row["all_cmps"][name] = per_query
            row["stat"][name] = {"mean": statistics.mean(per_query),
                                 "hit": statistics.mean(hit), "miss": statistics.mean(miss),
                                 "min": min(per_query), "max": max(per_query)}
            print(f"  {name:<18} {t * 1e6:>11.2f} мкс/запрос {c:>12.2f} сравнений")
        results.append(row)

    with open("res/lr4.json", "w", encoding="utf-8") as f:
        json.dump(results, f, ensure_ascii=False)
    make_plots(results)
```

5. Характеристики вычислительной системы

Параметр	Значение
Операционная система	Linux 6.18 (x86_64), виртуальная машина
Процессор	Intel Xeon, 2,10 ГГц, 1 виртуальное ядро
Оперативная память	4 ГБ
Интерпретатор	CPython 3.12.3
Способ замера времени	`time.perf_counter()`, перед каждым прогоном вызывается `gc.collect()`

Абсолютные значения времени зависят от машины; для сопоставления алгоритмов существенны отношения и характер роста. Для воспроизведения результатов на собственной системе достаточно запустить приведённый листинг.

6. Результаты измерений

6.1. Среднее время одного запроса

Время серии из $k = 2000$ запросов делилось на k; для каждого n выполнено 3 прогона, приведена медиана.

Таблица 2. Среднее время одного запроса, мкс

n	Последовательный	Двоичный	Интерполяционный
5 000	75,11	1,53	0,39
50 000	780,81	1,47	0,32
500 000	7 934,91	2,00	0,35

Увеличение n в 10 раз увеличивает время последовательного поиска в 10,4 и 10,2 раза — это $\Theta(n)$. Время двоичного поиска при росте n в 100 раз изменилось лишь в 1,3 раза (от 1,5

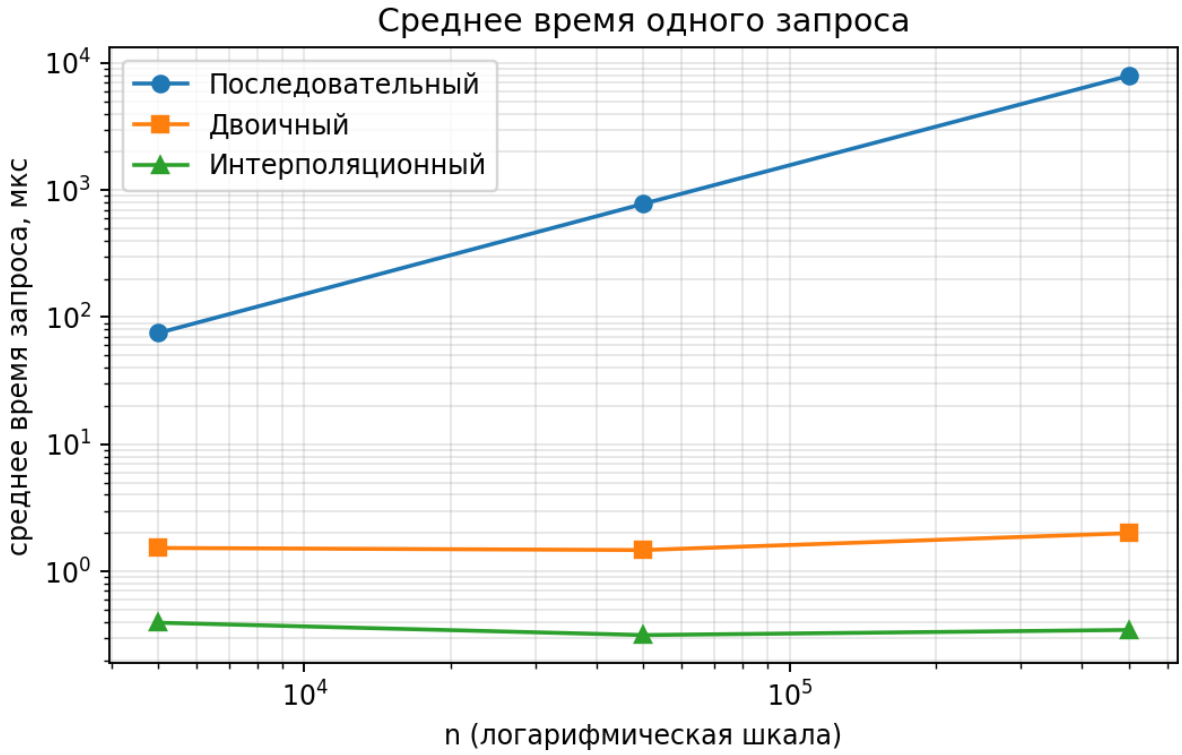


Рис. 1. Среднее время одного запроса (логарифмические шкалы)

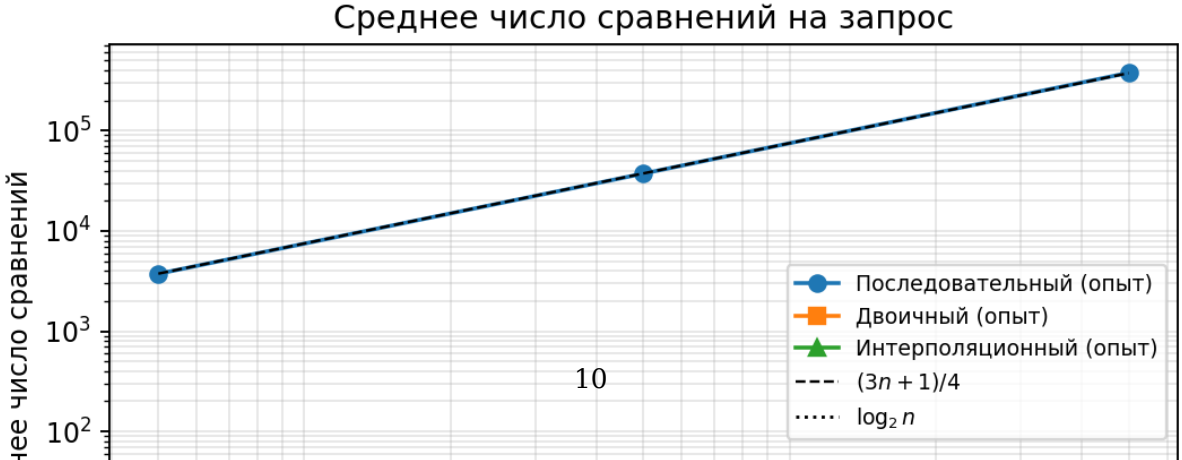
до 2,0 мкс). Малые абсолютные значения чувствительны к шуму: при повторных запусках значения для двоичного поиска колебались в пределах примерно 1–2,3 мкс, поэтому различия между соседними размерами в таблице находятся на уровне погрешности, и устойчивый вывод — лишь о том, что рост на два порядка по n меняет время на малую величину, как и должно быть при $\Theta(\log n)$. Интерполяционный поиск стабильно выполняется за 0,3–0,4 мкс независимо от n.

6.2. Среднее число сравнений

Таблица 3. Среднее число сравнений на один запрос (смесь 50/50)

n	Послед. (опыт)	Теория $\frac{3n+1}{4}$	Двоичный (опыт)	Теория (точная)	$\log_2 n$	Интерп. (опыт)	$\log_2 \log_2 n$
5 000	3 737,3	3 750,2	11,91	11,86	12,29	1,00	3,62
50 000	37 339,9	37 500,2	15,20	15,19	15,61	1,00	3,96
500 000	377 706,8	375 000,2	18,50	18,45	18,93	1,00	4,24

«Теория (точная)» для двоичного поиска — среднее значений $C_{\text{усп}}$ и $C_{\text{неусп}}$ из п. 2.2. Опытные значения лежат в пределах долей процента от теоретических для двоичного и интерполяционного поиска и в пределах 1 % для последовательного, где расхождение объясняется случайностью выборки из 1000 успешных запросов.



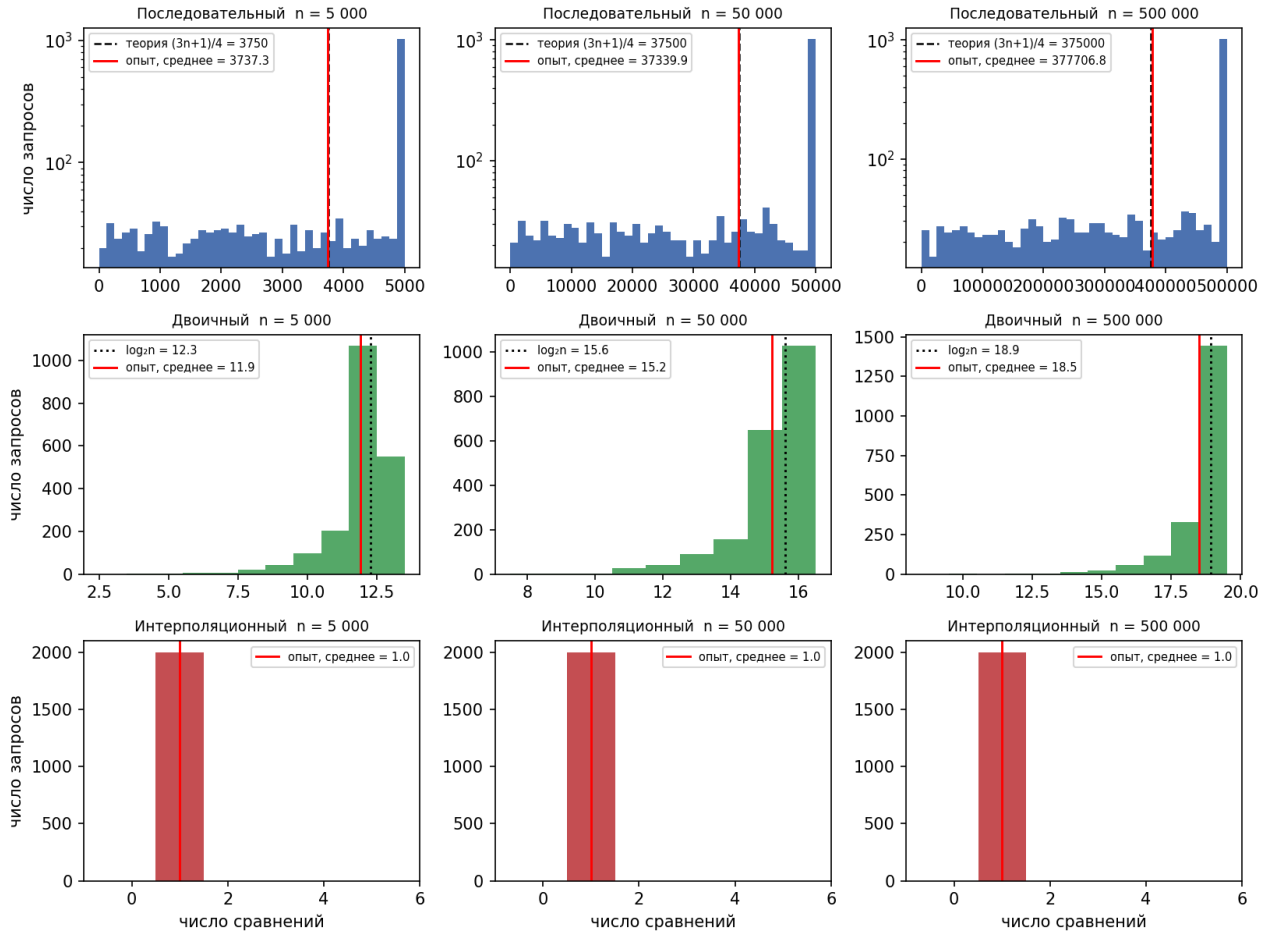


Рис. 3. Гистограммы числа сравнений по всем запросам

n	Алгоритм	Мин.	Макс.	Среднее
5 000	Последовательный	1	5000	3 737,32
	Двоичный	3	13	11,91
	Интерполяционный	1	1	1,00
50 000	Последовательный	13	50000	37 339,88
	Двоичный	8	16	15,20
	Интерполяционный	1	1	1,00
500 000	Последовательный	40	500000	377 706,81
	Двоичный	9	19	18,50
	Интерполяционный	1	1	1,00

Форма гистограмм:

- **последовательный поиск** — равномерное «плато» (успешные запросы распределены равномерно по значениям $1 \dots n$) и одиночный столбец в точке n, содержащий все 1000 неуспешных запросов;
- **двоичный поиск** — распределение сосредоточено у верхней границы $\lfloor \log_2 n \rfloor + 1$: половина узлов дерева находится на последнем уровне, поэтому большинство запросов требует максимального или почти максимального числа сравнений;
- **интерполяционный поиск** — единственный столбец в точке 1: все 2000 запросов обработаны за одно сравнение.

7.2. Сопоставление эмпирического среднего с теорией

Запросы смешанные, поэтому формулы $\frac{n+1}{2}$ (успешный поиск) и n (неуспешный) сопоставлены отдельно с соответствующими подвыборками.

Таблица 5. Эмпирическое среднее и теоретические значения

n	Алгоритм	Запросы	Опыт	Теория	Откл. от точной
5 000	Последовательный	успешные	2 474,6	$\frac{n+1}{2}$ = 2 500,5	−1,0 %
		неуспешные	5 000,0	n = 5 000	+0,0 %
	Двоичный	успешные	11,47	точно 11,36; $\log_2 n$ = 12,29	+0,9 %
		неуспешные	12,36	точно 12,36; $\log_2 n$ = 12,29	−0,0 %
	Интерполяционный	все	1,00	1 (прогрессия); $\log_2 \log_2 n$ = 3,62	—
50 000	Последовательный	успешные	24 679,8	$\frac{n+1}{2}$ = 25 000,5	−1,3 %
		неуспешные	50 000,0	n = 50 000	+0,0 %
	Двоичный	успешные	14,71	точно 14,69; $\log_2 n$ = 15,61	+0,1 %
		неуспешные	15,70	точно 15,69; $\log_2 n$ = 15,61	+0,0 %
	Интерполяционный	все	1,00	1 (прогрессия); $\log_2 \log_2 n$ = 3,96	—
500 000	Последовательный	успешные	255 413,6	$\frac{n+1}{2}$ = 250 000,5	+2,2 %
		неуспешные	500 000,0	n = 500 000	+0,0 %
	Двоичный	успешные	18,04	точно 17,95; $\log_2 n$ = 18,93	+0,5 %
		неуспешные	18,95	точно 18,95; $\log_2 n$ = 18,93	−0,0 %
	Интерполяционный	все	1,00	1 (прогрессия); $\log_2 \log_2 n$ = 4,24	—

Для последовательного поиска средние по успешным запросам отличаются от $\frac{n+1}{2}$ не более чем на 2,2 %, что соответствует разбросу выборки из 1000 запросов: стандартное отклонение среднего равно $\frac{n}{\sqrt{12}\sqrt{1000}} \approx 0{,}009\,n$, а наибольшее отклонение составило около $0{,}011\,n$, то есть не более 1,2 стандартного отклонения. Для неуспешных запросов среднее совпадает с n точно. Среднее двоичного поиска ниже $\log_2 n$ на 0,4–0,9 сравнения и совпадает с точными формулами п. 2.2 с погрешностью около 0,1 сравнения; значит, $\log_2 n$ следует понимать как оценку сверху (для успешного поиска фактическое значение близко к $\log_2 n - 1$).

8. Выводы

1. Эксперимент подтверждает теоретические оценки. Среднее число сравнений последовательного поиска совпало с $\frac{n+1}{2}$ для успешных запросов (в пределах разброса выборки) и с n для неуспешных, двоичного — с точными значениями, близкими к $\log_2 n$ (немного ниже него), интерполяционного — с $\Theta(1)$.

2. На упорядоченном массиве с ключами в виде арифметической прогрессии интерполяционный поиск оптимален: формула пробной позиции попадает точно в искомый элемент, и любой запрос, успешный или нет, обрабатывается за одно сравнение независимо от n. Это лучше, чем $O(\log \log n)$ для случайных равномерных ключей, поскольку у прогрессии нет случайных отклонений от линейной зависимости. При $n = 5 \cdot 10^5$ время запроса составило 0,35 мкс против 2,00 мкс у двоичного поиска (в 5,7 раза быстрее) и 7 935 мкс у последовательного (в 3 968 раз медленнее двоичного).
3. Этот результат нельзя переносить на произвольные данные: на кластеризованных и степенных распределениях (коды K и C) интерполяционный поиск деградирует вплоть до $\Theta(n)$, а регулярность прогрессии, при которой он идеален, в практических данных редка. Двоичный поиск гарантирует $\Theta(\log n)$ при любом распределении.
4. Гистограммы показывают различие между средним и худшим случаями: у последовательного поиска разброс числа сравнений охватывает весь диапазон $1 \dots n$, у двоичного он узок (9–19 сравнений при $n = 5 \cdot 10^5$ и среднем 18,5), у интерполяционного отсутствует.
5. Практически: если массив упорядочен и запросов много, замена последовательного поиска двоичным сокращает время в сотни и тысячи раз уже при $n = 5 \cdot 10^4$–$5 \cdot 10^5$ (в опыте — примерно в 530 и 4000 раз); затраты на предварительную сортировку (порядка $n \log n$) окупаются после числа запросов, растущего лишь логарифмически от n.